



HCM-UTE

TRƯỜNG ĐẠI HỌC

CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

HCMC University of Technology and Engineering

[SOEN330679] CÔNG NGHỆ PHẦN MỀM

CHƯƠNG 4: THIẾT KẾ GIAO DIỆN (02)

ThS. Võ Quang Phúc

MỤC TIÊU

1. **MT1.** Phân biệt các phương pháp đánh giá UI/UX: Hiểu rõ bản chất và cách áp dụng Heuristic Evaluation, Usability Testing, và A/B Testing.
2. **MT2.** Làm chủ quy trình Thiết kế Lặp (Iterative Design): Vận dụng thành thạo vòng lặp khép kín: Nghiên cứu → Thiết kế → Prototype → Kiểm thử → Cải tiến.
3. **MT3.** Phân cấp các sản phẩm thiết kế: Phân biệt vai trò và ứng dụng của Low-fi Wireframe, High-fi Mockup, và Interactive Prototype trong dự án.
4. **MT4.** Ứng dụng Component-driven Design: Hiểu cách xây dựng và sử dụng Design System để tối ưu hóa tốc độ phát triển phần mềm.
5. **MT5.** Thực hành Wireframe & Prototype: Tạo Lo-fi Wireframe cho các màn hình trọng tâm và dựng luồng tương tác (Navigation Flow) cho tính năng cốt lõi.
6. **MT6.** Triển khai Kiểm thử nhanh (Guerrilla Testing): Thực hiện phiên Usability Testing nhanh với 3–5 người dùng và biết cách tổng hợp kết quả cải tiến

NỘI DUNG

1. Thiết kế theo vòng lặp (Iterative design)
2. Heuristic Evaluation: Đánh giá bởi chuyên gia
3. Usability Testing: Kiểm thử người dùng thật
4. A/B Testing: So sánh định lượng
5. Component-driven Design và Design System trong thực tiễn
6. Quy trình thiết kế giao diện
7. Thực hành

1. Thiết kế theo vòng lặp (Iterative design)

1.1. Tổng quan

- ❑ Thiết kế giao diện không phải là công việc “làm một lần, dùng mãi mãi” mà là một quá trình tiến hóa và cải tiến liên tục.
- ❑ Sản phẩm được tinh chỉnh qua các chu kỳ ngắn khép kín: Nghiên cứu (Research) → Thiết kế (Design) → Prototype → Kiểm thử (Test).
- ❑ Mọi sự thay đổi và tối ưu hóa đều dựa trên dữ liệu thực tế từ người dùng, không dựa trên phán đoán chủ quan của đội ngũ phát triển.
- ❑ Sau mỗi chu kỳ, nhóm phát triển thu thập được bằng chứng thực nghiệm (Insight thực tế) để xác định chính xác những gì hoạt động tốt và những gì cần thay đổi.

1. Thiết kế theo vòng lặp (Iterative design)

1.2. Vòng lặp Iterative Design – 4 giai đoạn

① **NGHIÊN CỨU (Research):**

- ❑ Thu thập thông tin về người dùng mục tiêu, bối cảnh sử dụng, và vấn đề cần giải quyết.
- ❑ Các kỹ thuật: phỏng vấn người dùng, khảo sát (survey), phân tích dữ liệu sử dụng (analytics), nghiên cứu đối thủ cạnh tranh.
- ❑ Đây là giai đoạn xây dựng nền tảng → Thiết kế không có nghiên cứu là thiết kế dựa trên giả định.

1. Thiết kế theo vòng lặp (Iterative design)

1.2. Vòng lặp Iterative Design – 4 giai đoạn

② THIẾT KẾ (Design):

- ❑ Chuyển đổi hiểu biết từ nghiên cứu thành các giải pháp giao diện cụ thể.
- ❑ Giai đoạn này bắt đầu từ Wireframe thô (Lo-fi), dần dần tinh chỉnh thành Mockup chi tiết (Hi-fi).
- ❑ Không cố tạo ngay ra sản phẩm hoàn hảo, mục tiêu là tạo ra thứ gì đó đủ cụ thể để kiểm thử được.

1. Thiết kế theo vòng lặp (Iterative design)

1.2. Vòng lặp Iterative Design – 4 giai đoạn

③ PROTOTYPE:

- ❑ Xây dựng phiên bản tương tác đủ để người dùng sử dụng thử.
- ❑ Prototype không phải là sản phẩm thật, nó là công cụ học tập.
- ❑ Prototype giúp phát hiện vấn đề thiết kế trước khi đầu tư chi phí lập trình.
- ❑ Chi phí sửa một Prototype = 1/10 chi phí sửa trong code.

1. Thiết kế theo vòng lặp (Iterative design)

1.2. Vòng lặp Iterative Design – 4 giai đoạn

④ KIỂM THỬ (Test):

- ❑ Đưa Prototype cho người dùng thực tế sử dụng, quan sát và ghi nhận khó khăn họ gặp phải.
- ❑ Sau kiểm thử, nhóm phân tích kết quả, ưu tiên vấn đề theo mức độ nghiêm trọng, và bắt đầu chu kỳ mới.
- ❑ Không bao giờ ngừng tại giai đoạn này, luôn có thêm một vòng lặp để cải thiện.

2. Heuristic Evaluation: Đánh giá bởi chuyên gia

2.1. Quy trình Heuristic Evaluation (5 bước)

- ❑ **Bước 1 - Chuẩn bị:** Xác định phạm vi đánh giá (toàn bộ hệ thống hay một luồng cụ thể), chuẩn bị Prototype hoặc tài liệu mô tả giao diện, và chọn 3–5 evaluator (người đánh giá) có kiến thức về UI/UX.
- ❑ **Bước 2 - Đánh giá độc lập:** Mỗi evaluator làm việc riêng lẻ, không trao đổi với nhau. Mỗi người đi qua toàn bộ giao diện ít nhất hai lần: lần đầu để làm quen tổng thể, lần hai để tập trung phát hiện vấn đề với từng heuristic.
- ❑ **Bước 3 - Ghi nhận vấn đề:** Với mỗi vấn đề phát hiện được, ghi lại: (a) vị trí cụ thể trong giao diện, (b) heuristic nào bị vi phạm, (c) mô tả vấn đề, (d) điểm severity từ 0–4.

2. Heuristic Evaluation: Đánh giá bởi chuyên gia

2.1. Quy trình Heuristic Evaluation (5 bước)

- ❑ **Bước 4 - Tổng hợp (Aggregation meeting):** Các evaluator ngồi lại với nhau, trình bày danh sách vấn đề của từng người. Họ nhất danh sách, loại bỏ trùng lặp, và đạt đồng thuận về điểm severity cuối cùng cho từng vấn đề.
- ❑ **Bước 5 - Ưu tiên hóa và Báo cáo:** Sắp xếp vấn đề theo severity, phân chia thành 3 nhóm xử lý: ngay lập tức (severity 4), trước release (severity 3), backlog cải tiến (severity 1–2). Trình bày báo cáo cho team phát triển.

2. Heuristic Evaluation: Đánh giá bởi chuyên gia

2.2. Nghiên cứu về hiệu quả của Heuristic Evaluation

- ❑ Nghiên cứu kinh điển của Nielsen (1992) chỉ ra rằng:
- ❑ Một evaluator đơn lẻ chỉ phát hiện được khoảng 35% vấn đề trong một giao diện.
- ❑ Khi tăng lên 5 evaluator, tỷ lệ phát hiện tăng lên 75%.
- ❑ Đây là lý do tại sao Heuristic Evaluation được thực hiện bởi nhóm chuyên gia (minimum 3 người), không phải một cá nhân.

3. Usability Testing: Kiểm thử người dùng thật

3.1. Tổng quan

- ❑ Trái ngược với Heuristic Evaluation (dựa vào chuyên gia), phương pháp này quan sát trực tiếp hành vi của những người sẽ thực sự sử dụng sản phẩm trong đời sống hàng ngày.
- ❑ Là phương pháp có độ tin cậy cao nhất (Ecological Validity) vì thu thập dữ liệu từ hành vi thật, loại bỏ hoàn toàn các phán đoán hay giả định chủ quan của đội ngũ phát triển.
- ❑ Nguyên lý cốt lõi (Steve Krug): “Nếu muốn sản phẩm hoạt động tốt hơn, hãy kiểm thử nó với người dùng thực. Kiểm thử sớm và thường xuyên.”

3. Usability Testing: Kiểm thử người dùng thật

3.2. Các hình thức Usability Testing

Moderated Testing (Có điều phối viên)

- ❑ Điều phối viên ngồi cùng người dùng (trực tiếp hoặc online qua Zoom), hướng dẫn và đặt câu hỏi trong khi người dùng thực hiện các tác vụ.
- ❑ Ưu điểm: có thể hỏi sâu về lý do.
- ❑ Nhược điểm: tốn thời gian, có thể ảnh hưởng đến hành vi người dùng.
Phù hợp: giai đoạn early-stage design.

3. Usability Testing: Kiểm thử người dùng thật

3.2. Các hình thức Usability Testing

Unmoderated Remote Testing (Không có điều phối viên, từ xa)

- ❑ Người dùng sử dụng Prototype tại nhà riêng theo thời gian họ chọn, màn hình và âm thanh được ghi lại tự động.
- ❑ Công cụ: UserTesting.com, Maze, Lookback.
- ❑ Ưu điểm: tiết kiệm thời gian, dữ liệu nhiều hơn.
- ❑ Phù hợp: giai đoạn validation với Prototype đã tương đối hoàn thiện.

3. Usability Testing: Kiểm thử người dùng thật

3.2. Các hình thức Usability Testing

Guerrilla Testing (Kiểm thử du kích)

- ☐ Tiếp cận người lạ tại quán cafe, thư viện hoặc sảnh trường và nhờ họ thử sản phẩm trong 10–15 phút.
- ☐ Không cần lên lịch trước, không cần lab.
- ☐ Chi phí: zero.
- ☐ Ưu điểm: cực kỳ nhanh và rẻ.
- ☐ Cần cẩn thận: người được chọn phải thuộc nhóm người dùng mục tiêu.

3. Usability Testing: Kiểm thử người dùng thật

3.2. Các hình thức Usability Testing

A/B Testing (Kiểm thử phân tách)

- ❑ Hai phiên bản giao diện (A và B) được chạy song song cho hai nhóm người dùng khác nhau.
- ❑ Kết quả được so sánh bằng dữ liệu định lượng (tỷ lệ click, conversion rate, thời gian hoàn thành tác vụ).
- ❑ Phù hợp: sản phẩm đang production với lượng người dùng đủ lớn (ít nhất vài nghìn).

4. A/B Testing: So sánh định lượng

4.1. Định nghĩa

- ❑ A/B Testing (hay Split Testing) là phương pháp so sánh hai phiên bản thiết kế bằng cách đo lường hiệu quả thực tế của từng phiên bản trên người dùng thật, dựa trên dữ liệu định lượng, không phải ý kiến chủ quan. Đây là công cụ mạnh mẽ của các sản phẩm đang vận hành ở quy mô lớn.
- ❑ Mỗi lần chỉ thay đổi **MỘT biến số**. Nếu phiên bản B thay đổi cả màu nút, văn bản nút, vị trí nút và kích thước ảnh so với phiên bản A, bạn không thể biết yếu tố nào tạo ra sự khác biệt. A/B Testing chỉ có giá trị khi kiểm soát chặt chẽ các biến số.

4. A/B Testing: So sánh định lượng

4.2. So sánh các phương pháp

Phương pháp	Ai thực hiện?	Khi nào dùng?	Kết quả dạng?
Heuristic Evaluation	Chuyên gia UI/UX (3–5 người)	Mọi giai đoạn, từ sớm	Danh sách vấn đề + severity
Usability Testing (Guerrilla)	Người dùng mục tiêu (5–8 người)	Khi đã có Prototype	Observation + insight định tính
Usability Testing (Lab)	Người dùng mục tiêu (có kiểm soát)	Giai đoạn pre-release	Metrics + observation
A/B Testing	Người dùng thật (hệ thống live)	Sản phẩm đang production	Dữ liệu định lượng (% , tỷ lệ)

5. Component-driven Design và Design System trong thực tiễn

5.1. Vấn đề khi không có Design System: “CSS is a Mess”

- ❑ Hãy tưởng tượng một dự án web có 5 lập trình viên cùng phát triển song song. Mỗi người có xu hướng viết CSS/style riêng theo phong cách của mình.
- ❑ Sau 3 tháng: nút 'Đăng nhập' ở trang A có padding 8px, ở trang B có padding 12px; màu xanh primary ở màn hình 1 là #2E75B6, ở màn hình 2 là #2D7DC4; font size của heading ở mỗi trang là mỗi người đặt khác nhau.
- ❑ Kết quả cuối cùng trông như một sản phẩm được ghép từ nhiều sản phẩm khác nhau → inconsistent, thiếu chuyên nghiệp, và khó bảo trì.

5. Component-driven Design và Design System trong thực tiễn

5.2. Component-driven Design

- ❑ Component-driven Design là triết lý thiết kế giao diện bằng cách xây dựng từ các thành phần tái sử dụng (reusable components), thay vì thiết kế từng màn hình như một trang đơn lẻ.
- ❑ Mỗi component là một đơn vị UI độc lập, có trạng thái (state) rõ ràng, hành vi nhất quán, và có thể được dùng lại ở bất kỳ đâu trong hệ thống.

5. Component-driven Design và Design System trong thực tiễn

5.2. Component-driven Design

Ví dụ: Thay vì thiết kế 'màn hình đăng nhập', 'màn hình đặt hàng' và 'màn hình tài khoản' riêng lẻ, ta thiết kế:

- ❑ một component 'Button' với 3 variants (Primary, Secondary, Danger),
- ❑ một component 'TextInput' với các trạng thái (Default, Focused, Error, Disabled),
- ❑ một component 'Card' với nhiều kích thước.

Các màn hình sau đó chỉ là sự kết hợp của các component này theo bố cục phù hợp.

5. Component-driven Design và Design System trong thực tiễn

5.2. Component-driven Design

Cấp độ Component	Mô tả	Ví dụ thực tế
Atoms (Nguyên tử)	Đơn vị nhỏ nhất, không thể chia nhỏ hơn.	Button, TextInput, Icon, Label, Color token, Typography scale
Molecules (Phân tử)	Kết hợp 2–3 atoms tạo thành một đơn vị có chức năng.	Search bar (Input + Button + Icon), Form field (Label + Input + Error message)
Organisms (Sinh vật)	Kết hợp nhiều molecules tạo thành một khu vực giao diện.	Navigation bar, Product card, Comment section, Checkout form
Templates (Khuôn mẫu)	Bố cục trang với các vùng placeholder, chưa có nội dung thật.	Dashboard layout, Detail page layout, Settings page layout
Pages (Trang)	Template được điền nội dung thật, sản phẩm hoàn chỉnh.	Trang chủ Shopee, Chi tiết sản phẩm, Trang thanh toán

5. Component-driven Design và Design System trong thực tiễn

5.2. Component-driven Design

- ❑ Hệ thống phân cấp này được gọi là **Atomic Design**, đề xuất bởi Brad Frost (2016).
- ❑ Đây là nền tảng cho hầu hết mọi Design System hiện đại, từ Material Design (Google) đến Ant Design (Alibaba) đến Shopify Polaris.

5. Component-driven Design và Design System trong thực tiễn

5.3. Lợi ích định lượng của Design System

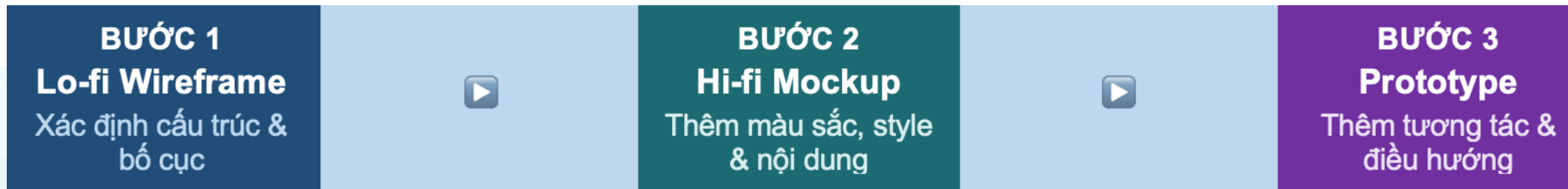
Theo báo cáo của Figma (2023), các tổ chức áp dụng Design System báo cáo:

- ❑ Tăng 34% tốc độ thiết kế: Thử nghiệm của Figma chứng minh Designer đạt mục tiêu nhanh hơn rõ rệt nhờ kéo-thả linh kiện (Component) có sẵn thay vì vẽ lại từ đầu.
- ❑ Tối ưu hóa thời gian Hand-off: Lập trình viên không phải “đoán” thông số (padding, margin, color); giảm thiểu tối đa thời gian giao tiếp nhờ hệ thống tài liệu (Documentation) tường minh.
- ❑ Triệt tiêu “Nợ thiết kế” (Design Debt): Đảm bảo tính nhất quán (Consistency). Khi cập nhật (ví dụ: thay đổi Color Token), toàn bộ hệ thống tự động đồng bộ trên mọi màn hình.

6. Quy trình thiết kế giao diện

6.1. Tổng quan

- ❑ Quy trình thiết kế giao diện từ yêu cầu ban đầu đến sản phẩm tương tác được chia thành ba cấp độ rõ ràng, mỗi cấp có mục tiêu, công cụ, và tiêu chí hoàn thành riêng.
- ❑ Ba cấp độ này không phải là ba bước tuần tự một chiều mà đi qua chúng trong nhiều vòng lặp. Tuy nhiên, sự phân tách này giúp người thiết kế biết rõ mình đang tập trung vào cái gì ở từng giai đoạn.



6. Quy trình thiết kế giao diện

6.2. Lo-fi Wireframe: Bản phác thảo cấu trúc

- ❑ Bản phác thảo tổng thể tối giản: Chỉ sử dụng các hình khối cơ bản (hình chữ nhật, đường thẳng) và văn bản giả định (Lorem Ipsum) để định vị các thành phần giao diện.
- ❑ Nguyên tắc “3 KHÔNG”: Không màu sắc, không hình ảnh thật, không font chữ đẹp → tập trung hoàn toàn vào cấu trúc và bố cục thay vì yếu tố thẩm mỹ.

6. Quy trình thiết kế giao diện

6.2. Lo-fi Wireframe: Bản phác thảo cấu trúc

- ❑ Tâm lý “chưa hoàn thiện”: Giao diện trông thô sơ giúp người dùng, khách hàng và đồng đội thoải mái đưa ra phản hồi thẳng thắn mà không ngại “phá vỡ” công sức của Designer.
- ❑ Tối ưu hóa quá trình sửa đổi: Thúc đẩy các cuộc thảo luận cốt lõi về luồng đi và tính năng (ví dụ: vị trí đặt nút bấm, thứ tự thông tin) một cách nhanh chóng trước khi bước vào giai đoạn thiết kế chi tiết.

6. Quy trình thiết kế giao diện

6.2. Lo-fi Wireframe: Bản phác thảo cấu trúc

Yếu tố	Cách thể hiện trong Lo-fi
Bố cục trang (Layout)	Hình chữ nhật xác định các vùng: header, sidebar, content area, footer. Chỉ cần tỷ lệ đúng, không cần chính xác pixel.
Thanh điều hướng (Navigation)	Hình chữ nhật ngang ở trên cùng với các nhãn tab bằng text thô: 'Menu', 'Đơn hàng', 'Báo cáo'.
Nội dung (Content)	Placeholder text '[Tên sản phẩm]', '[Giá]', đường kẻ ngang thay cho paragraph text, hình chữ nhật có dấu X ở giữa thay cho ảnh.
Hành động (Actions/Buttons)	Hình chữ nhật có bo góc với label: 'Nút CTA chính', 'Nút Hủy'. Không có màu — chỉ cần xác định vị trí.
Form/Input	Hình chữ nhật nằm ngang với label phía trên: 'Tên món', 'Số lượng', 'Ghi chú'.
Dữ liệu dạng bảng/danh sách	Grid placeholder với text [Mục 1], [Mục 2]... hoặc các hàng kẻ ngang đại diện cho danh sách.

6. Quy trình thiết kế giao diện

6.2. Lo-fi Wireframe: Bản phác thảo cấu trúc

Công cụ vẽ Lo-fi Wireframe

- ❑ Bút và giấy (Paper Prototyping): Nhanh nhất cho ý tưởng đầu tiên. Phù hợp để phác thảo trong cuộc họp nhóm.
- ❑ draw.io (diagrams.net): Miễn phí hoàn toàn, hoạt động trên trình duyệt, có thể export PNG/PDF.
- ❑ Figma (Wireframe mode): Tài khoản miễn phí cho tối đa 3 project. Mạnh mẽ hơn draw.io và cũng dùng được cho Hi-fi Mockup và Prototype.
- ❑ Balsamiq Mockups: Công cụ chuyên dụng cho Lo-fi, có phong cách 'sketchy'. Có bản dùng thử 30 ngày.

6. Quy trình thiết kế giao diện

6.3. Hi-fi Mockup: Bản thiết kế chi tiết

- ❑ High-fidelity Mockup (Mockup độ trung thực cao) là bước chuyển từ cấu trúc sang hình thức thị giác hoàn chỉnh.
- ❑ Một Hi-fi Mockup trông gần giống với sản phẩm thực tế, bao gồm: màu sắc chính xác theo Design System, typography đúng loại và kích thước, spacing nhất quán, hình ảnh và icon thật (hoặc placeholder chất lượng cao), và trạng thái của từng UI element.
- ❑ Hi-fi Mockup là hình ảnh tĩnh, người xem nhìn vào nhưng chưa tương tác được.

6. Quy trình thiết kế giao diện

6.3. Hi-fi Mockup: Bản thiết kế chi tiết

Các quy tắc thiết yếu khi tạo Hi-fi Mockup

- ❑ **Sử dụng Color System nhất quán:** Xác định trước: Primary color, Secondary color, Success, Warning, Error, và Neutral palette. Sử dụng đúng màu này cho đúng mục đích trên tất cả màn hình. Không tự ý dùng màu 'gần giống' mà không có lý do.
- ❑ **Quy tắc 8pt Grid:** Tất cả spacing (margin, padding, gap) nên là bội số của 8px (8, 16, 24, 32, 48, 64px). Quy tắc này đảm bảo sự nhất quán về khoảng cách và giúp bàn giao cho lập trình viên dễ dàng hơn.

6. Quy trình thiết kế giao diện

6.3. Hi-fi Mockup: Bản thiết kế chi tiết

Các quy tắc thiết yếu khi tạo Hi-fi Mockup

- ❑ **Thiết kế mọi trạng thái (States):** Mỗi UI element cần có đủ trạng thái: Default, Hover, Active/Pressed, Disabled, Error, Loading (nếu phần tử có thời gian chờ). Thiếu bất kỳ trạng thái nào cũng là thiếu sót thiết kế.
- ❑ **Không bỏ qua Empty States:** Màn hình danh sách trống trông như thế nào? Màn hình kết quả tìm kiếm không có kết quả? Empty state tốt là cơ hội để giao tiếp với người dùng và hướng dẫn họ hành động tiếp theo.
- ❑ **Accessibility Checklist:** Đảm bảo contrast ratio của text trên nền đạt ít nhất 4.5:1 (WCAG AA). Có thể kiểm tra online tại contrast-ratio.com hoặc plugin Figma 'Contrast'. Đây là yêu cầu pháp lý ở nhiều quốc gia.

6. Quy trình thiết kế giao diện

6.4. Interactive Prototype: Giao diện tương tác

- ❑ Prototype là bước cuối cùng trước khi bàn giao cho lập trình viên, và là công cụ quan trọng nhất để kiểm thử với người dùng.
- ❑ Một Interactive Prototype cho phép người dùng thực sự tương tác: nhấn nút, điều hướng giữa màn hình, cuộn trang, xem animation - tất cả mà không cần một dòng code nào.

6. Quy trình thiết kế giao diện

6.4. Interactive Prototype: Giao diện tương tác

Click-through Prototype (Prototype click đơn giản)

- ❑ Kết nối các màn hình Hi-fi Mockup bằng các hotspot (vùng click).
- ❑ Người dùng nhấn vào một vùng → chuyển sang màn hình tiếp theo.
- ❑ Phù hợp: kiểm thử luồng điều hướng cơ bản.
- ❑ Công cụ: Figma Prototype mode, Adobe XD.

6. Quy trình thiết kế giao diện

6.4. Interactive Prototype: Giao diện tương tác

Micro-interaction Prototype (Prototype tương tác chi tiết)

- ❑ Thêm animation và transition: menu mở bằng slide-down, nút nhấn có ripple effect, form validation hiện inline error.
- ❑ Phù hợp: demo cho khách hàng, kiểm thử UX chi tiết.
- ❑ Công cụ: Figma Smart Animate, Principle, ProtoPie.

6. Quy trình thiết kế giao diện

6.4. Interactive Prototype: Giao diện tương tác

High-fidelity Functional Prototype (Prototype chức năng đầy đủ)

- ❑ Prototype kết nối với API thật hoặc mock data.
- ❑ Người dùng có thể nhập dữ liệu thật, thấy kết quả thật.
- ❑ Phù hợp: demo final với stakeholder, pitch deck.
- ❑ Công cụ: Framer, Webflow, hoặc code HTML/CSS/JS đơn giản.

THỰC HÀNH